

Smart Attendance and Classroom Access Platform with Face Verification and Geofenced Check-In

PID 12

Test Plan Report

Version 1.0

Mentor: Mr. Pasan Sarathchandra

Group Members

230702K - WICKRAMAARACHCHI W.A.A.U.

230714A - WIJAYARATHNE T.W.O.A.

230734J - WITHANA W.D.P.S.

230736R - WITHANAGE M.H.

Contribution

Group Member	Sections written	Testing performed
230702K - WICKRAMAARAC HCHI W.A.A.U.	1. Evaluation Mission 3.1.1 Database Integrity 3.1.6 Security and Access Control 4.1 Core API and security results	Core API test suite and recorded its coverage. 11 database integration tests skipped in the standard run. 11-endpoint access control matrix and recorded its results.
230714A - WIJAYARATHNE T.W.O.A.	2. Target Test Items 3.1.3 User Interface Testing 3.1.8 Configuration Testing 4.1 Web, API and accessibility results	Web dashboard test suite and recorded its coverage. API validation and authorization using Swagger UI. Lighthouse audits for the administrator and lecturer dashboards.
230734J - WITHANA W.D.P.S.	3.1.2 Function Testing 3.1.4 Performance Profiling 4.1 Face-verification and performance results	Face-verification test suite and recorded its coverage. Prepared the positive and negative functional test cases. Ran and analysed the k6 single-user performance scenarios.
230736R - WITHANAGE M.H.	3.1.5 Load Testing 3.1.7 Failover and Recovery 4.2 Test Coverage 5. Risks and Constraints 4.1 Android and load results	Android test suite and recorded its coverage. Ran and analysed the k6 authentication load test. Prepared the requirements traceability matrix and documented the planned failover tests.

Table of Contents

1. Evaluation Mission and Test Motivation.....	3
2. Target Test Items.....	4
2.2 Components built by the team.....	4
2.2 Third-party components the system depends on.....	5
2.3 Hardware and environment.....	5
3. Test Approach.....	6
3.1 Testing Techniques and Types.....	9
3.1.1 Data and Database Integrity Testing.....	9
3.1.2 Function Testing.....	11
3.1.3 User Interface Testing.....	15
3.1.4 Performance Profiling.....	18
3.1.5 Load Testing.....	23
3.1.6 Security and Access Control Testing.....	28
3.1.7 Failover and Recovery Testing.....	31
3.1.8 Configuration Testing.....	34
4. Deliverables.....	36
4.1 Test Evaluation Summaries.....	36
4.2 Reporting on Test Coverage.....	43
5. Risks, Dependencies, Assumptions, and Constraints.....	45
6. References.....	48

1. Evaluation Mission and Test Motivation

UniAttend is a smart attendance platform for university lectures. A lecturer opens an attendance session from a web dashboard, and students mark their attendance from a mobile application. Before a student is counted present, the system checks several independent pieces of evidence, whether the student is physically inside the classroom geofence, the student's face matches an enrolled reference face, the face capture passes a liveness challenge, and, while the lecture runs, the student scans a rotating QR code displayed by the lecturer. When the lecturer closes the session, the system weighs that evidence and writes a final record of present, late or absent for every student on the roster.

This mission makes testing unusually important for three reasons.

First, the system produces an academic record. An attendance record affects a student's eligibility to sit an examination. A false negative, where a student who attended is marked absent, harms that student directly. A false positive, where a student who did not attend is marked present, defeats the purpose of the system. The cost of a defect here is higher than in an ordinary web application.

Second, the system is adversarial by design. Students have a motive to defeat it. The realistic attacks are a shared QR screenshot, a photograph of an absent friend's face held to the camera, and a replayed or tampered request sent directly to the API. Testing therefore has to include deliberate attempts to defeat the controls, not only checks that the happy path works.

Third, the system handles biometric data. Face embeddings derived from students' faces are personal data. They are stored encrypted, and access to them must be restricted. A confidentiality failure here is more serious than a functional bug.

The system is built as a monorepo of four deployable parts (a Next.js web application, an Expo / React Native Android application, a FastAPI core API and a separate FastAPI face-verification service) behind a Keycloak identity provider, with PostgreSQL and Redis. Because the parts are separate, a large share of the risk sits at the boundaries between them, so the test approach covers each part on its own and then the assembled system.

The main objectives of the testing process are listed below.

- Find defects before the platform is used in a real lecture. Attendance is recorded once, in real time, in front of a class. There is no convenient opportunity to repeat a lecture because the system failed, so defects have to be found before deployment rather than during use.
- Verify that the system meets its functional requirements. Every use case (session creation and closing, check-in, manual attendance, manual review, correction requests, reporting and administration) is exercised with valid, invalid and boundary data, and traced back to the requirement it implements.
- Verify the non-functional requirements. These include response time and stability when a full lecture hall checks in within about a minute, the accuracy of the face verification and

geofence decisions, the usability of both interfaces, accessibility, and the battery and memory cost of the Android application during a lecture.

- Assess the security of the platform against the OWASP Top 10, with particular attention to broken access control, since the system has three roles (student, lecturer, administrator) whose data must not leak into each other.
- Assess the resilience of the platform. The core API depends on a separate face service, a database and Redis. Testing establishes what the system does when one of those is unavailable mid-check-in, and confirms that no partial or duplicate attendance record is left behind.
- Advise on the residual quality risks. Where a risk cannot be eliminated within the project timeline, it is recorded in section 5 with a mitigation, rather than left unstated.

2. Target Test Items

The listing below identifies those test items (software, hardware and supporting product elements) that have been identified as targets for testing. The list covers both components built by the project team and third-party components the system depends on.

2.2 Components built by the team

Core API (services/core-backend): A FastAPI application in Python that holds almost all of the business logic: session lifecycle, QR session generation and validation, geofence evaluation, check-in decisions, attendance finalization, manual attendance and review, correction requests, reporting, administration and push notification dispatch. It is the single most important test item, because a defect here affects both client applications.

Face-verification service (services/face-verification): A separate FastAPI service that enrolls reference faces, generates and compares face embeddings using InsightFace, validates liveness evidence produced on the device, and stores embeddings in encrypted form. It is tested separately because it is deployed separately, it is the most computationally expensive component, and its decisions are probabilistic rather than exact.

Lecturer and administrator web application (apps/web): A Next.js application used by lecturers to create and manage sessions, display the rotating QR code, watch live attendance, perform manual attendance and review, and export reports; and by administrators to manage users, classrooms, academic data, attendance policy, institution reports and the audit log.

Student Android application (apps/mobile): An Expo / React Native application used by students to sign in, see their courses and upcoming sessions, check in to a session (location, liveness challenge, face capture, QR scanning), and manage notification preferences. Android only for this release.

Database schema and migrations (database/): The PostgreSQL schema, the forward migrations and their rollback scripts, including the migrations that introduced encrypted face

embeddings and removed the previous plaintext embedding column..

Deployment configuration (deployment/, docker-compose.production.yml) The production Compose file, environment validation scripts, the reverse proxy configuration, and the backup and restore scripts for Keycloak and the application database.

2.2 Third-party components the system depends on

Component	Role in the system	Why it is a test item
Keycloak	Identity provider; issues the access tokens presented by both the web and mobile clients	Authentication and role assignment depend entirely on it
PostgreSQL (Supabase)	Application database	Holds the attendance record; integrity and concurrency behaviour matter
Redis	Temporary session and verification state	Check-in behaviour when it is unavailable must be defined
InsightFace through ONNX Runtime	Face embedding and comparison	Determines the accuracy of the face decision
ML Kit face detection (on device)	Liveness challenge observation	Determines whether a photograph can defeat liveness
Expo push notification service	Delivery of reminders and alerts	Delivery is outside the team's control and must degrade safely
Reverse proxy and TLS	Public access to all four services	Certificate and routing failures take the whole system down

2.3 Hardware and environment

Item	Detail
Server	Single VPS running the four services under Docker Compose behind a TLS reverse proxy
Test Android device	Pixel 6 pro, Android 17, 12GB RAM

Development and test machine	Windows 11, Python 3.11.9, Node.js 24.x, Docker
Browsers	Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)
Network	Campus Wi-Fi and mobile data, to reflect real classroom conditions

3. Test Approach

The test approach presents the strategy used to design and implement the required tests. It is organised around two ideas.

The first is a test pyramid. The largest number of tests are fast automated tests that run on every push through GitHub Actions and verify individual services and components in isolation, using fakes in place of Keycloak, Redis, the face service and the push-notification provider. Above them sit integration tests against a real PostgreSQL database, then end-to-end and exploratory testing through the real user interfaces on a real device. Defects are cheapest to find at the bottom of the pyramid, so that is where the bulk of the tests live; but a test at the bottom cannot prove the assembled system works, which is why the upper layers exist.

The second idea is that each verification control is tested twice: once for whether it accepts what it should, and once for whether it rejects what it should. A geofence test that only proves a student inside the classroom is accepted is half a test. The other half is proving a student outside is refused, and that a student standing on the boundary gets a consistent result according to the configured rule.

A summary of the approach for each technique follows. Each is expanded in section 3.1.

Data and Database Integrity Testing

The schema is PostgreSQL, accessed through asyncpg and repository classes rather than an ORM. The repositories are tested directly against a migrated database, away from the user interface. Particular attention is given to the constraints that stop a student being marked present twice, and to the behaviour of concurrent check-in requests. Every migration is applied and rolled back on a clean database.

Function Testing

Black-box testing is performed for each use case through the API and user interfaces using valid, invalid, and boundary data. The automated test suites verify functionality at the service level, while manual API testing is performed through the Swagger UI in the local development environment. Exploratory testing on the Android device and in the web browser

is used to evaluate the complete user flows.

User Interface Testing

Covers both clients. The web dashboard is tested in three browsers and at several screen widths. The Android application is tested on a physical device, because camera, GPS and permission behaviour cannot be reproduced faithfully on an emulator. Accessibility is assessed with Lighthouse for the web, and with TalkBack and contrast checks for the application. Usability is assessed with a small moderated study and the System Usability Scale.

Performance Profiling

Response times are measured per endpoint under a single user to establish a baseline, then under load. The face-verification service is profiled separately, since InsightFace inference is CPU-bound and is the expected bottleneck. On the device, memory is profiled with Android Studio Profiler and battery drain is measured with adb shell dumpsys batterystats over a full simulated lecture.

Load Testing

The realistic worst case is a lecture hall where every student checks in within about sixty seconds of the session opening. Load is applied in steps against the deployed API, and the system is observed for error rate, response time, database connection exhaustion and, most importantly, for correctness under concurrency: no duplicate attendance rows, and no attendance granted without its evidence.

Security and Access Control Testing

Structured around the OWASP Top 10, with broken access control as the priority. A role matrix is built listing every endpoint against the three roles, and every disallowed combination is attempted. Authentication is tested with expired, tampered and foreign-issuer tokens. Verification controls are tested using QR replay attempts, controlled mock-location tests and printed/screen-displayed images against the implemented liveness challenge. Dependency and container scans are also performed.

Failover and Recovery Testing

Dependencies are removed one at a time from a local copy of the stack, not from the deployed system, and the resulting behaviour is recorded. The test is not only that an error is shown, but that the database is left in a consistent state and that retrying after recovery succeeds. Database backup and restore testing was outside the current test scope because the deployed project uses the Supabase Free Plan and a separate recovery environment was not available.

Configuration Testing

The Android application is tested across device sizes and Android versions, and under denied or revoked camera and location permissions, with GPS switched off, and on both Wi-Fi and

mobile data. The web application is tested across browsers and viewport sizes. The server is tested under both the development and the production Compose configurations.

3.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

The database and its processes are tested as an independent subsystem, without the user interface as the route to the data. The application uses PostgreSQL through asyncpg, with SQL written in repository classes, so the tests exercise those repositories directly against a migrated database.

Technique Objective	Exercise the database access methods and the schema constraints independently of the user interface, so that incorrect persistence, lost updates and data corruption can be observed and logged. In particular, establish that the attendance record cannot be duplicated, that evidence cannot outlive the session it belongs to, and that every migration can be applied and reversed on a clean database.
Technique	• Run the repository-level integration suites against a dedicated local PostgreSQL instance, seeded from the project schema and migrations, using the <code>*_TEST_DATABASE_DSN</code> environment variables the suites expect. • Invoke each repository method with valid and invalid data, then inspect the resulting rows directly with SQL rather than through the API. • Issue concurrent check-in requests for the same student and session and confirm that exactly one attendance row results and that the attempt numbering stays unique. • Apply every migration in database/migrations in order to an empty database, then apply each rollback script, and confirm the schema returns to its prior state. • Confirm that face embeddings are stored only in encrypted form. • Verify foreign keys and cascade behaviour by attempting to delete referenced rows (a classroom bound to a session, a student on a roster).
Oracles	The rows in the database, read directly with SQL, are the oracle. Each integration test asserts against the persisted state rather than the API response, so a response that claims success while writing nothing is detected. For concurrency, the oracle is a count: the number of attendance rows after N simultaneous requests must be exactly one. For the migration tests, the oracle is a schema comparison before and after the rollback.

Required Tools	• pytest with pytest-asyncio, for the integration suites • DBeaver, for direct inspection • Supabase dashboard, for observing the deployed database
Success Criteria	• All repository integration suites pass against a real PostgreSQL instance. • Concurrent check-in produces exactly one attendance record. • Every migration applies and rolls back cleanly on an empty database. • No plaintext face embedding exists in any table. • No orphaned rows remain after a referenced entity is removed.
Special Considerations	• These tests must never be pointed at the deployed Supabase database. They create, mutate and delete data, and several of them deliberately provoke constraint violations. A local throwaway instance is used instead. • Eleven integration tests are skipped when the corresponding DSN environment variable is absent, which is the default. They must be run deliberately; a green suite without them is not evidence that the database layer is correct. • A small, controlled data set is used so that an unexpected extra row is visible rather than lost in volume.

3.1.2 Function Testing

Function testing focuses on requirements that trace directly to use cases and business rules. The goal is to verify correct data acceptance, processing and retrieval, and correct application of the business rules. It is black-box testing, performed through the API and through the two user interfaces, and supported by the automated suites at service level.

Technique Objective	Exercise every use case of the platform, including navigation, data entry, processing and retrieval, and observe and log the behaviour. Verify that the system makes the correct attendance decision for both normal and unusual combinations of verification results.
Technique	Execute each use-case scenario with valid, invalid and boundary data, and verify that the expected result occurs for valid data, that a clear and specific error is shown for invalid data, and that each business rule is applied. The scenarios are grouped as follows. Session lifecycle. Create a session; activate it before, inside and after its scheduled window; start and stop a QR batch; close the session; attempt to close it a second time; cancel a session with and without a reason. Check-in. Geofence inside, outside, and exactly on the boundary including the accuracy buffer; face score above, at and below the configured similarity threshold; liveness evidence that is valid, malformed, expired, future-dated or marked as failed; a QR value that is current, from the previous rotation window, from another session, or replayed after use; check-in before the session opens and after it closes. Finalization. Confirm that closing a session writes exactly one record per roster student, that the present, late and absent decisions follow the recorded evidence, and that a student with no evidence is marked absent rather than omitted. Manual attendance and manual review. A lecturer marks a student manually with a reason; a reason is required; a borderline face result is approved or rejected in review. Correction requests. A lecturer submits a request; an administrator approves or rejects it; the attendance record changes accordingly and an audit row is written. Administration. Users, classrooms, academic data and attendance policy are created, edited and removed; institution reports and CSV exports produce correct figures; the audit log records the action.

	Notifications. A device token is registered and re-registered; preferences are honoured; an upcoming-class reminder is produced at the right time. The automated test suites cover these rules at service and route level. API-visible cases are manually tested through the FastAPI Swagger UI in the local environment. The remainder are executed manually through the web dashboard and the Android application.
Oracles	For the automated suites, the assertions in the tests. For manual API testing, the documented expected status code and response body for each request, compared against the actual response in Swagger UI, together with the resulting database row. For interface testing, the requirement and the visible outcome: the attendance state shown to the student must match the record the lecturer sees and the record stored in the database. A disagreement between those three views is a defect even if each view is individually plausible.
Required Tools	• pytest, for the core API and face-verification suites • Vitest with Testing Library, for web components and server actions • Jest with jest-expo, for Android application logic • Swagger UI / OpenAPI specification, used to derive the test cases and confirm the contract • GitHub Actions, which runs the suites on every push and pull request • A physical Android device and the deployed web dashboard
Success Criteria	• Every use case in the requirements has at least one positive and one negative test case, recorded in the traceability matrix. • All automated suites pass. • Every API endpoint returns the documented status code and body for valid input, and a specific, non-generic error for invalid input. • The attendance outcome is identical in the student application, the lecturer dashboard and the database for every test case.

Special Considerations	- Three roles exist, and several use cases are available to only one of them. Test accounts for student, lecturer and administrator are required, and each case is executed as the role that owns it. - Check-in has ordering constraints: QR is deliberately not a precondition of the initial check-in but is weighed at finalization. Test cases must reflect the intended rule rather than an assumed one. - Manual testing against the deployed system writes real rows to the production database. A dedicated test course, test session and test students are used, and the data is removed or marked afterwards.
-------------------------------	--

Status and result:

Functional Requirement	Positive Test Case	Test Evidence	Status
Authenticate a user and return their identity	test_returns_the_signed_in_application_user	core-backend/tests/test_me_route.py	Pass
Lecturer creates an attendance session	test_create_session	core-backend/tests/test_lecturer_sessions_route.py	Pass
Validate student location during check-in	test_eligible_student_pass_is_stored_and_remains_in_progress	core-backend/tests/test_geofence_service.py	Pass
Verify a student's QR code	test_verify_qr_session_accepts_correct_qr_value	core-backend/tests/test_qr_session_service.py	Pass
Verify a student's face with valid liveness evidence	test_valid_liveness_allows_face_verification_through_backend_chain	core-backend/tests/test_attendance_face_verification_route.py	Pass

Finalize attendance when a session closes	test_closing_decides_every_students_attendance_from_the_real_evidence	core-backend/tests/test_attendance_lifecycle_database_integration.py	Pass
Lecturer records manual attendance	test_sets_the_status_and_returns_the_manual_record	core-backend/tests/test_manual_attendance_route.py	Pass
Administrator reviews correction requests	test_pending_request_can_be_approved_and_is_audited	core-backend/tests/test_correction_request_review.py	Pass

Table 1: Positive Functional Test Cases

Functional Requirement	Negative Test Case	Test Evidence	Status
Authenticate a user and return their identity	test_rejects_an_expired_token	core-backend/tests/test_me_route.py	Pass
Lecturer creates an attendance session	test_create_session_invalid_schedule_returns_422	core-backend/tests/test_lecturer_sessions_route.py	Pass
Validate student location during check-in	test_a_failed_geofence_reading_does_not_ask_check_in_to_run	core-backend/tests/test_geofence_service.py	Pass
Verify a student's QR code	test_verify_qr_session_returns_expired_for_expired_token	core-backend/tests/test_qr_session_service.py	Pass
Verify a student's face with valid liveness evidence	test_missing_liveness_is_rejected_through_the_backend_chain	core-backend/tests/test_attendance_face_verification_route.py	Pass

Finalize attendance when a session closes	test_closing_twice_is_refused_and_writes_nothing_more	core-backend/tests/test_attendance_lifecycle_database_integration.py	Pass
Lecturer records manual attendance	test_a_student_cannot_set_attendance	core-backend/tests/test_manual_attendance_route.py	Pass
Administrator reviews correction requests	test_lecturer_cannot_list_or_delete	core-backend/tests/test_correction_request_review.py	Pass

Table 2: Negative Functional Test Cases

3.1.3 User Interface Testing

User interface testing verifies the user's interaction with the software, for both the lecturer and administrator web dashboard and the student Android application. It covers navigation and access, the behaviour of individual controls, conformance to usability and accessibility standards, and the quality of the experience under realistic conditions.

Technique Objective	Exercise both interfaces to observe and log standards conformance and behaviour, specifically: - Navigation reflects the user's role, and no route is reachable that the role may not use. - Every control (button, form field, dialog, table, filter, export) behaves as labelled, and its disabled, loading, empty and error states are reachable and correct. - The web dashboard renders correctly in Chrome, Firefox and Edge and at desktop, laptop and tablet widths. - The Android application renders correctly on the physical test device, and remains usable in a lecture hall, meaning one-handed, on a possibly dim screen, in limited time. - Error messages tell the user what happened and what to do next, rather than exposing an internal error. - Both interfaces meet WCAG 2.1 AA for contrast, focus visibility, target size and screen-reader labelling.
----------------------------	--

Technique	Automated component testing. The web components and server actions are covered by Vitest and Testing Library, including the dialogs for manual attendance, review, session lifecycle and account provisioning. The Android application logic is covered by Jest. Manual navigation and functional testing. Each page of the dashboard and each screen of the application is walked through against a checklist: correct heading, correct data, working links, working back navigation, working form submission, and a sensible empty state when there is no data. State testing. Each screen is observed in its loading, empty, error and populated states, by disconnecting the network, by using an account with no courses, and by forcing a server error. Cross-browser and responsive testing. The dashboard is opened in Chrome, Firefox and Edge, and at 1920, 1366 and 768 pixel widths. Accessibility testing. Lighthouse is run on each major dashboard page. Colour contrast is checked against the WCAG AA ratios. The Android application is navigated with TalkBack on the check-in flow and the dashboard screen. Usability study. 20-25 participants who have not used the system attempt three tasks each (sign in and find today's lecture; complete a check-in; as a lecturer, open a session and export the roster). Time on task, errors and requests for help are recorded, and each participant completes the ten-item System Usability Scale questionnaire.
Oracles	For automated component tests, the assertions in the tests. For manual testing, the interface design and the requirement: a screen is correct when it shows the data the requirement says it should, in a state the user can act on. For accessibility, the WCAG 2.1 AA success criteria and the Lighthouse reports. For usability, the task completion rate, the observed errors and the SUS score, where a score above 68 is the accepted industry average. Screenshots record the observation, because an interface defect is easier to demonstrate than to describe.

Required Tools	• Vitest and Testing Library (web), Jest and jest-expo (Android) • Google Chrome, Mozilla Firefox, Microsoft Edge, and their developer tools • Lighthouse (in Chrome DevTools) • A colour contrast checker • TalkBack on the Android device • A physical Android device, Pixel 6 Pro • System Usability Scale questionnaire, administered as a form
Success Criteria	• All automated component tests pass. • Every page and screen loads without a console error and without a broken route. • Every form reports validation failures next to the field responsible, before submission where possible. • The dashboard renders without horizontal scrolling or overlapping content in all three browsers at all three widths. • Lighthouse accessibility score of 90 or above on each major page. • All usability participants complete all three tasks, and the mean SUS score is 68 or above.
Special Considerations	• The camera, GPS and permission behaviour of the Android application cannot be tested faithfully on an emulator. All device testing is performed on a physical handset. • The application is Android only in this release. iOS is out of scope and is recorded in section 5. • Parts of the interface depend on live data (the rotating QR code, the live attendance list), so those screens are tested during an actual open session rather than in isolation. • Usability participants are given the task and not the route, since telling a participant where to click destroys the value of the observation.

3.1.4 Performance Profiling

Performance profiling measures response times, throughput and resource consumption, and evaluates them against defined targets. Two environments are profiled: the server, where the check-in path must stay responsive while a class arrives, and the Android device, where the application must not exhaust battery or memory during a lecture.

Targets are defined before measurement, so that a result can be judged rather than merely reported.

Measurement	Target
User login / Authentication	under 2 seconds end to end
Dashboard page load	under 2 seconds end to end
Active Attendance Sessions	under 2 seconds end to end
Face Verification	under 10 seconds end to end

Table 3: Response-Time Targets for Key Operations

Technique Objective	Measure response time, transaction consistency and server resource usage for important UniAttend operations under a controlled single-user workload. The profile covers: ● Mobile PKCE login ● Mobile dashboard ● Active attendance-session retrieval ● Face verification using test image The purpose is to establish a reliable single-user performance baseline before load testing with multiple virtual users.
----------------------------	--

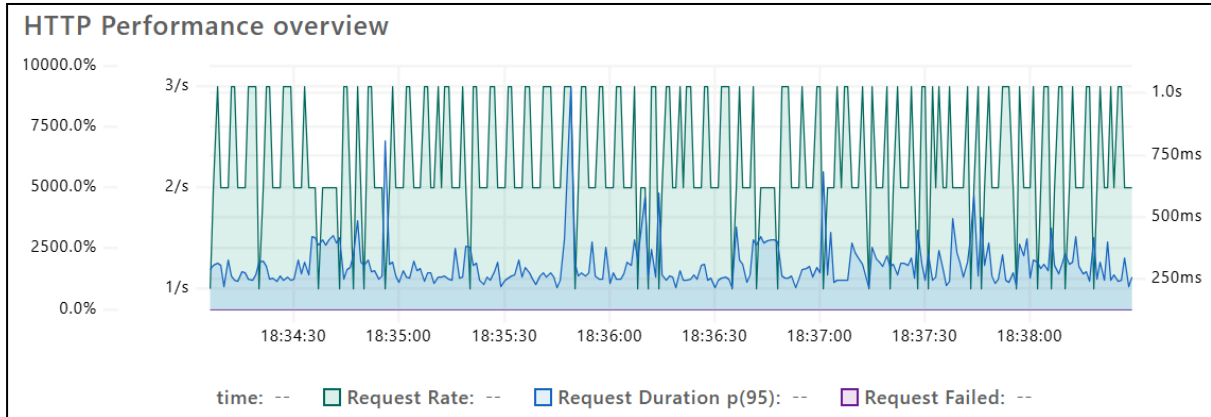
Technique	Use Grafana k6 scripts executed from a Windows workstation against the deployed VPS through the production HTTPS endpoints. Each scenario used: • One virtual user • 200 measured transactions • Sequential execution • k6 built-in dashboard export • Functional checks for every response • CPU and RAM monitoring on the VPS during execution The face-verification test uploaded the protected JPEG image associated with the enrolled test account. The InsightFace model remained loaded in the container throughout the test.
Oracles	The following checks were used to determine pass or failure: • HTTP response status is successful. • Login returns a valid authorization code and access token. • Student profile contains a valid registration number. • Active sessions response is a valid array. • Face readiness response contains a valid readiness status. • Face-verification response returns HTTP 200 and a processing status. • The supplied face image is accepted as a match for the enrolled test account. • No request or functional check fails. • Response-time thresholds remain within the defined limits. • No container restarts during testing. • VPS memory remains stable and swap is not consumed

Required Tools	• k6 v2.3.0 • k6 built-in web dashboard • k6 HTML dashboard export • SSH access to the Netcup VPS • Docker container statistics • Linux <code>top</code>, <code>free</code> and disk monitoring commands • Protected test user accounts • Protected face test image • Existing PostgreSQL database and deployed application services
Success Criteria	The performance profile is successful when: • All 200 transactions in each scenario complete successfully. • Functional check failure rate is 0%. • Face-verification match rate is 100% for the enrolled test image. • p95 response times remain below: ○ PKCE login: 3 seconds ○ Student profile: 1.5 seconds ○ Active sessions: 1.5 seconds ○ Face readiness status: 2 seconds ○ Face verification: 10 seconds • No container restarts or out-of-memory events occur. • Swap remains unused. • Test dashboards and JSON summaries are generated successfully. • CPU and RAM evidence is collected during the test.
Special Considerations	• This is a single-user performance profile, not a concurrent capacity test. • One virtual user was intentionally used to measure individual transaction behaviour consistently. • Multiple-user behaviour will be evaluated separately during load testing. • Requests still passed through the real public HTTPS endpoints, so workstation-to-VPS network time is included. • The face model was kept running and was not restarted between transactions.

- The face test reused one protected JPEG image, so it measures inference consistency rather than variation across different poses, lighting conditions or camera devices.

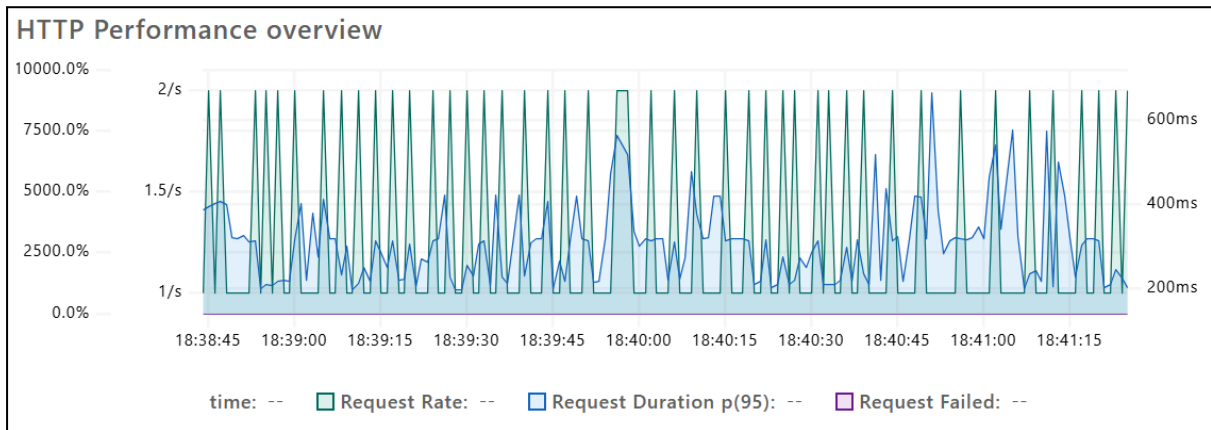
Status and result:

1. User login / Authentication



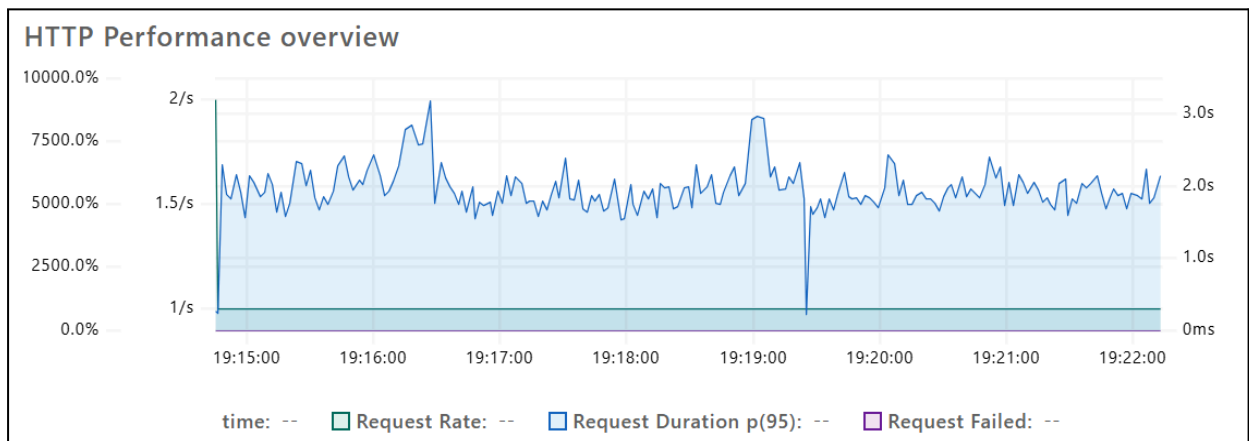
Trends							
metric	avg	max	med	min	p90	p95	p99
http_req_blocked	890µs	533ms	0ms	0ms	0ms	0ms	0ms
http_req_connecting	338µs	202ms	0ms	0ms	0ms	0ms	0ms
http_req_duration	273ms	1s	253ms	200ms	353ms	409ms	509ms
http_req_receiving	4ms	822ms	438µs	0ms	6ms	10ms	59ms
http_req_sending	295µs	1ms	408µs	0ms	605µs	710µs	801µs
http_req_tls_handshaking	531µs	318ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	268ms	713ms	251ms	198ms	345ms	407ms	496ms
iteration_duration	1s	2s	1s	1s	1s	1s	1s
login_duration	824ms	1s	771ms	671ms	1s	1s	1s

2. Dashboard page load



Trends							
metric	avg	max	med	min	p90	p95	p99
http_req_blocked	10ms	886ms	0ms	0ms	0ms	0ms	446ms
http_req_connecting	5ms	470ms	0ms	0ms	0ms	0ms	203ms
http_req_duration	292ms	665ms	283ms	196ms	421ms	468ms	577ms
http_req_receiving	111µs	1ms	0ms	0ms	507µs	833µs	1ms
http_req_sending	161µs	1ms	0ms	0ms	435µs	504µs	528µs
http_req_tls_handshaking	5ms	415ms	0ms	0ms	0ms	0ms	233ms
http_req_waiting	291ms	665ms	283ms	196ms	420ms	468ms	577ms
iteration_duration	793ms	1s	774ms	696ms	921ms	973ms	1s
student_profile_duration	289ms	665ms	274ms	196ms	421ms	470ms	577ms

3 . Face Verification



Trends							
metric	avg	max	med	min	p90	p95	p99
face_verification_duration	1s	3s	1s	1s	2s	2s	2s
http_req_blocked	8ms	464ms	0ms	0ms	0ms	0ms	434ms
http_req_connecting	4ms	219ms	0ms	0ms	0ms	0ms	200ms
http_req_duration	1s	3s	1s	220ms	2s	2s	2s
http_req_receiving	297µs	23ms	0ms	0ms	616µs	1ms	4ms
http_req_sending	6ms	536ms	1ms	0ms	1ms	1ms	2ms
http_req_tls_handshaking	4ms	253ms	0ms	0ms	0ms	0ms	231ms
http_req_waiting	1s	3s	1s	215ms	2s	2s	2s
iteration_duration	2s	3s	2s	1s	2s	2s	3s

Docker Container CPU and Memory Usage During Face-Verification Profiling

r20955f565348	uniattend-app-keycloak-db-1	0.14%	45.56MiB / 768MiB	5.93%
cddbba5e56de6	uniattend-app-redis-1	0.57%	9.637MiB / 256MiB	3.76%
f9e541ada3988	uniattend-database-application-db-1	0.12%	49.73MiB / 1GiB	4.86%
dc73e3868bb449	uniattend-app-keycloak-1	0.31%	627.4MiB / 1.75GiB	35.01%
10ffc9cc095b7	uniattend-app-core-api-1	0.17%	62.82MiB / 768MiB	8.18%
cdfea053e20033	uniattend-face-face-verification-1	169.09%	815MiB / 2GiB	39.80%
ac93d0fe317925	uniattend-app-web-1	0.01%	183.9MiB / 768MiB	23.95%

3.1.5 Load Testing

Load testing subjects the system to varying workloads to measure how it behaves as the workload grows, and to establish that it continues to function correctly beyond the expected maximum. For this platform the realistic worst case is precise: a lecture hall in which every student attempts to check in within roughly sixty seconds of the lecturer opening the session. Unlike most load testing, correctness matters as much as speed here, because the failure mode that matters is not a slow response but a wrong attendance record produced under concurrency.

Technique Objective	Evaluate whether the UniAttend deployment continues to operate correctly under expected, peak and stress workloads. Measure transaction response time, throughput, failure rate, CPU consumption, memory consumption, swap usage and container stability. The completed phase concentrated on the mobile authentication flow through Keycloak. It evaluated both instantaneous login bursts and a sustained arrival rate. Later phases will cover student dashboard traffic, attendance check-in bursts, QR verification, Face Verification concurrency and a short stability workload.									
Technique	Apache k6 virtual users executed the complete mobile authorization-code-with-PKCE login flow: 1. Request the Keycloak authorization page. 2. Submit the student’s credentials. 3. Receive an authorization code. 4. Exchange the code for access and refresh tokens. 5. validate the HTTP responses and issued access token. Three hundred dedicated Keycloak load-test accounts were created. Each virtual user received a separate account to represent independent students. <table><tr><th>Workload</th><th>Configuration</th></tr><tr><td>Small instantaneous peak</td><td>20 virtual users starting together</td></tr><tr><td>Expected class-size peak</td><td>50 virtual users starting together</td></tr><tr><td>High instantaneous peak</td><td>100 virtual users starting together</td></tr></table>		Workload	Configuration	Small instantaneous peak	20 virtual users starting together	Expected class-size peak	50 virtual users starting together	High instantaneous peak	100 virtual users starting together
Workload	Configuration									
Small instantaneous peak	20 virtual users starting together									
Expected class-size peak	50 virtual users starting together									
High instantaneous peak	100 virtual users starting together									

	Stress peak	200 virtual users starting together
	Sustained arrival workload	200 login arrivals distributed across one minute
	“Starting together” means that k6 activated all virtual users at approximately the beginning of the scenario. Operating-system scheduling and network activity caused requests to arrive across a short time interval rather than the exact same millisecond. The sustained test was configured for 200 arrivals per minute. k6 completed 201 iterations because one iteration was scheduled at the one-minute boundary. Future load tests will reuse the same staged approach for: • Student profile, active-session and notification reads • Geofence check-in and QR attendance bursts using unique students • Face Verification with progressively increasing concurrency • Mixed student traffic over a short stability period	
Oracles	k6 automatically determined login success by checking that: • The authorization page returned HTTP 200. • The Keycloak login form was present. • Credential submission produced a valid redirect. • An authorization code was returned. • The token endpoint returned HTTP 200. • An access token was issued. The following acceptance thresholds were applied: • Login failure rate below 1%. • Login p95 response time below 3,000 milliseconds. • No container restarts or out-of-memory termination. • No swap usage during the workload. • All services remained healthy after testing.	

	The k6 HTML dashboards and JSON summaries provide transaction evidence. Docker and host monitoring provide CPU, RAM, swap and restart evidence. For future attendance tests, database counts and uniqueness constraints will verify that every virtual student receives the correct attendance result without missing or duplicated records.
Required Tools	● Apache k6 v2.3.0 for workload generation and automated validation ● k6 built-in HTML dashboard for visualization ● Three hundred generated Keycloak load-test accounts ● PowerShell for test orchestration ● SSH for VPS monitoring ● Docker statistics for per-container CPU and memory ● Linux <code>top</code> and <code>free</code> for host CPU, RAM and swap
Success Criteria	A workload passes when: ● The workload completes without test-script errors. ● At least 99% of login transactions succeed. ● Login p95 remains below 3 seconds. ● No container restarts or becomes unhealthy. ● The host does not use swap. ● Memory remains within configured limits. ● The system recovers to its normal operating state after the workload.

Special Considerations	• Testing was performed against the deployed Netcup VPS with 4 vCPU and approximately 7.8 GiB RAM. • PostgreSQL was hosted on the same VPS. • Requests travelled through the public HTTPS endpoint, so workstation-to-VPS network latency is included. • All 300 accounts were synthetic Keycloak users. • Background service health checks continued during testing. • The highest sampled host CPU usage was 69.6%. • Keycloak reached approximately 129.23% Docker CPU, equivalent to about 1.29 CPU cores. • Keycloak peak memory was approximately 707.6 MiB. • Host maximum used memory was approximately 3,097 MiB. • Swap usage and container restarts remained zero. • This phase covers authentication load only. Attendance, dashboard, Face Verification concurrency and stability workloads remain planned for later execution.
-------------------------------	--

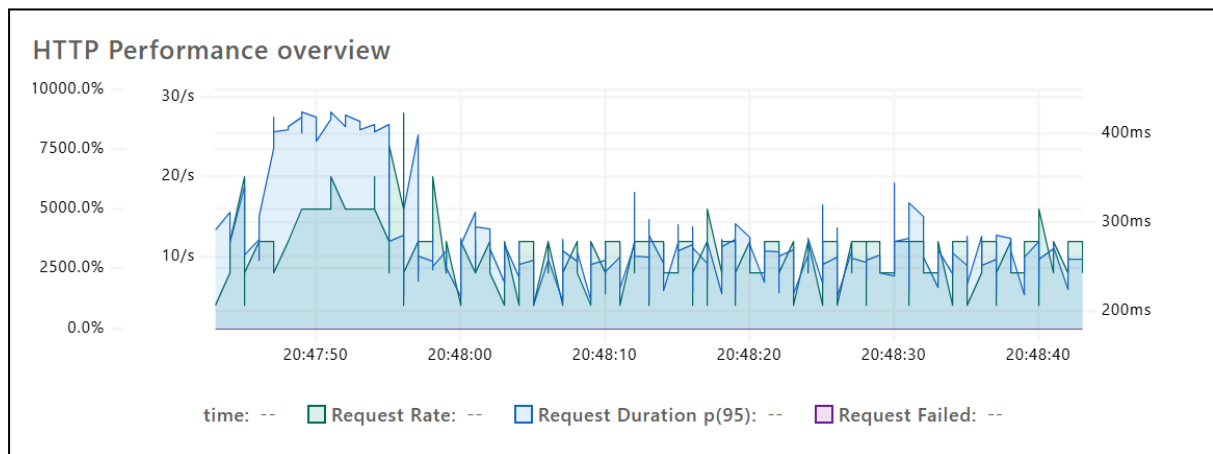
Status and result:

1. Login/Authentication Load testing

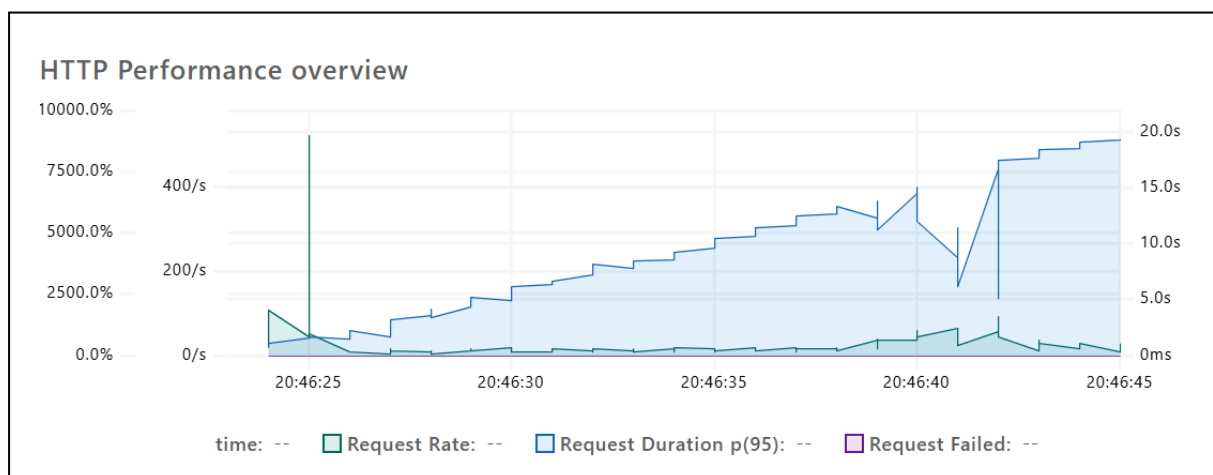
Summary

scenario	target_logins	completed_logins	failures	min_ms	average_ms	median_ms	p95_ms	max_ms	result
Simultaneous 20	20	20	0	1357	1884.6	1946	2321.05	2322	Pass
Simultaneous 50	50	50	0	1586	3074.44	3168	4248.9	4377	Fail
Simultaneous 100	100	100	0	5434	7033.94	6466	9550	9944	Fail
Simultaneous 200	200	200	0	15569	18622.56	18557	21893.40	22561	Fail
Controlled 200/min	200	201	0	661	955.89	723	2098	2507	Pass

Controlled 200/min requests (3.3 Avg requests per second):



Stressed 200 requests :



3.1.6 Security and Access Control Testing

Security and access control testing focuses on two areas. Application-level security concerns access to data and business functions: a student may see and act on only their own attendance, a lecturer only on their own courses and sessions, and an administrator on institution-wide data. System-level security concerns access to the system itself: authentication through Keycloak, transport security, exposure of administrative surfaces, and the protection of biometric data at rest.

The platform has a further, unusual security requirement. Its users have a motive to defeat it, so testing includes deliberate attempts to record attendance without attending.

The tests are organised against the OWASP Top 10, with broken access control treated as the highest priority because the system's three roles share one API.

Technique Objective	Exercise the platform under the following conditions and observe and log the behaviour. • Application-level security. An actor can reach only the functions and data their role permits, and only their own records within that role. A lecturer cannot read or modify another lecturer's session; a student cannot read another student's attendance; a student cannot reach any administrative function. • System-level security. Only authenticated actors holding a valid token from the expected issuer are served. Administrative and documentation surfaces are not publicly exposed. Data in transit is encrypted, and face embeddings at rest are encrypted. • Control integrity. The verification controls cannot be satisfied without actually satisfying them: a QR value cannot be replayed or shared, a location cannot be fabricated, and a photograph cannot pass the liveness challenge.
----------------------------	---

Technique	Access control matrix. Every endpoint is listed against the three roles, and each cell is marked allowed or denied according to the requirement. Every denied cell is then attempted with a valid token for that role, and the response must be 403. Every allowed cell is attempted to confirm it is not accidentally blocked. This produces the largest single block of security test cases. Object-level access (IDOR). Authenticated as one student, the identifiers of another student's session, attendance record and correction request are substituted into otherwise valid requests. Authenticated as one lecturer, another lecturer's course and session identifiers are substituted. The expected result is a denial, not a record belonging to someone else. Authentication. Requests are sent with no token, an expired token, a token whose signature has been altered, a token issued by a different realm or issuer, a token for a user who has since been deleted, and a token whose role claim has been edited. Each must be rejected, and the rejection must not reveal why in a way that helps an attacker. Attacks on the verification controls. • QR: a value captured in one rotation window is submitted in a later window; a value captured on one device is submitted from another account; a value from a different session is submitted. • Location: a mock-location application is used on the test device to report classroom coordinates from outside the building. • Face and liveness: a photograph of an enrolled face is displayed on a second screen and on paper to the camera, and a recorded video of the challenge is replayed. Input validation and injection. Malformed and hostile input is submitted to every endpoint that accepts free text, coordinates or identifiers: SQL metacharacters, oversized payloads, out-of-range coordinates, and script content in fields that later appear in the dashboard or in a CSV export (including formula-injection payloads in exported cells). Automated scanning. OWASP ZAP is run in baseline (passive) mode against the deployed web application, and in active mode only against a local copy of the stack.
------------------	---

	Dependency and container scanning. npm audit --omit=dev for the JavaScript applications, pip-audit for the two Python services, and Trivy against the built Docker images. Secret and configuration review. The repository history is checked for committed credentials; the API documentation endpoints are checked on the deployed system; TLS configuration is checked.
Oracles	For access control, the matrix is the oracle: each cell has an expected status code, and any deviation is a defect whose severity is judged by what was exposed. For authentication, the expected outcome is a 401 or 403 with no sensitive detail in the body. For the control-integrity attacks, the oracle is the attendance record: the attack succeeded if an attendance record exists for a student who was not present, which is a high-severity defect regardless of the response code returned. For scanning, the ZAP, audit and Trivy reports, each finding triaged by whether it is reachable in the deployed configuration rather than accepted at the scanner's stated severity.
Required Tools	• Swagger UI, for the access control matrix and the authentication cases • jwt.io, for inspecting and altering tokens • OWASP ZAP, for passive and active scanning • npm audit, pip-audit and Trivy, for dependency and image scanning • Keycloak administration console, for creating the three test accounts and inspecting role mappings • A mock-location application on the physical Android device • A printed photograph and a second screen, for the liveness attack • git log -S over the repository, for committed secrets

Success Criteria	• Every denied cell in the access control matrix returns 403, and no response body contains another user's data. • All object-level access attempts are refused. • Every invalid, expired, tampered or foreign-issuer token is rejected. • No attendance record is produced by any of the control-integrity attacks. Where an attack succeeds, it is recorded as a high-severity defect with the mitigation stated. • API documentation endpoints return 404 on the deployed system. • No plaintext face embedding is retrievable from the database or through the API. • Every critical and high dependency finding is either patched or recorded with a justification for accepting it.
Special Considerations	• Only systems owned by the project are tested. No scanning or attack traffic is directed at university infrastructure or at any third-party service. • ZAP's active scan sends large volumes of malformed requests and creates spurious data. It is run only against a local copy of the stack, and only after a database backup. The deployed system receives the passive scan only. • Some findings cannot be remedied within the project timeline. Those are recorded in section 5 with a severity and a proposed mitigation rather than omitted. • Screenshots used as evidence are cropped or redacted so that no bearer token, password or connection string appears in the report. • Defeating liveness with a photograph is a realistic attack for this system and is tested directly rather than assumed to be prevented.

3.1.7 Failover and Recovery Testing

Failover and recovery testing establishes that the platform recovers from hardware, software and network failures without loss of data or loss of data integrity. The platform is composed of four services (Core Backend, Face Verification service, Lecturer/Admin Web application, and Student Android application) plus a database, a cache and an identity provider, so the

realistic failures are not power cuts but the loss of one dependency while the others continue to run, often in the middle of a student's check-in.

Technique Objective	Simulate failure conditions and exercise the recovery processes, manual and automatic, so that the platform, the database and the clients return to a known good state. The conditions included are: • the face-verification service unavailable or timing out during a check-in; • Redis unavailable while sessions and verification state are held in it; • the database unreachable, or the connection dropped mid-transaction; • Keycloak unavailable, so that no new token can be issued and no existing token can be validated against the key set; • the core API restarted while requests are in flight; • the student's device losing network connectivity part-way through a check-in; • the session closing while a student is mid-check-in;
Technique	Existing functional tests are repeated; the failure is introduced while they run. • Each dependency is stopped in turn with docker compose stop, a check-in is attempted, and the behaviour of the API and of the Android application is recorded. The dependency is then restarted and the same check-in is retried. • The core API container is killed with requests in flight, then restarted. • Device connectivity is removed by enabling flight mode part-way through the check-in flow, then restored. • The lecturer closes the session while a student is on the face capture screen. In every case the check is made at three levels: the error shown to the user, the state left in the database, and whether a retry after recovery succeeds and produces exactly one record.

Oracles	The database is the primary oracle: after a failure and recovery, there must be no partially written check-in, no attendance record whose supporting evidence is missing, and no duplicate record created by the retry. The second oracle is the user-visible message, which must tell the student that the check-in did not complete and that they should try again, rather than reporting success or displaying an internal error. The third is the application and container logs, which must record the failure with enough detail to diagnose it.
Required Tools	• Docker and Docker Compose, for stopping and restarting individual services • docker compose logs, for observing the failure • Supabase SQL Editor / Database Dashboard for inspecting the state left behind • The physical Android device, for the connectivity cases
Success Criteria	• Every dependency failure produces a clear, specific message to the user and no partial write to the database. • Retrying after the dependency is restored succeeds and produces exactly one attendance record. • A session closing mid-check-in is handled explicitly, with the student told that the session has closed.
Special Considerations	• These tests are performed against a local copy of the stack, not against the deployed system. Stopping services on the deployed system would interrupt anyone using it, and a restore test on the deployed database risks data loss. • Backup and restore testing is not included because a separate recovery environment and verified restore procedure were not used during this testing. • The classical recovery scenarios, such as disconnecting cabling or powering down a host, are not feasible on a hosted VPS. Container-level failure injection is used instead. • Only one dependency is intentionally made unavailable at a time so that the cause of any observed failure can be identified clearly.

3.1.8 Configuration Testing

Configuration testing verifies that the platform operates correctly across the hardware and software configurations it will meet in use. For the Android application this means different devices, Android versions, permission states and network conditions. For the web dashboard it means different browsers and screen sizes. For the server it means the development and production deployment configurations.

Technique Objective	Test the system under different supported configurations and conditions to check whether any feature becomes slow, behaves incorrectly, or fails. Special attention is given to camera and location permissions because these are required for student check-in. The application should also behave correctly if the user denies or later revokes these permissions.
Technique	Android devices. The application is installed and the full check-in flow is executed on the primary physical device, and repeated on a second device covering a different screen size and Android version. Permissions. The check-in flow is attempted with the camera permission denied; with the location permission denied; with location granted as approximate rather than precise; with a permission revoked from system settings while the application is running; and with the permission granted after an initial refusal. In each case the application must explain what is needed and offer a route to grant it, rather than failing silently or crashing. Device settings. Check-in is attempted with GPS switched off, and with the device in battery saver mode Network. The flow is executed on campus Wi-Fi, on mobile data, and on a deliberately slow connection, and with the connection lost and restored mid-flow. Browsers and viewports. The dashboard is exercised in Chrome, Firefox and Edge at 1920, 1366 and 768 pixel widths, covering the session, review, reporting and administration pages. Server configuration. Change administrator-configurable settings such as the geofence radius, face similarity threshold, and QR refresh interval, and verify that the system uses the updated values correctly.

Oracles	The functional test cases already defined are the oracle: a configuration passes when those cases produce the same outcome they produce in the reference configuration. For permission and device-service cases, the oracle is the message shown and the absence of a crash. For the configurable policy values, the oracle is the observed decision boundary: raising the geofence radius must widen the accepted area by the amount configured.
Required Tools	• The primary physical Android device, Pixel 6 Pro, Android 17, and a second device where available • Android Debug Bridge (adb), for installing builds and reading logs • Android system settings, for permission and location-service manipulation • Chrome, Firefox, Edge and their device-emulation modes • Docker Compose
Success Criteria	• The full check-in flow completes on every tested device. • No denied or revoked permission causes a crash; each produces an explanatory message and a route to recovery. • The dashboard is fully usable in all three browsers at all three widths. • The production configuration starts successfully without configuration errors. • Changing a configured policy value changes the system's decision accordingly, with no hard-coded fallback overriding it.
Special Considerations	• This release supports Android only. iOS is out of scope for now. • The number of physical devices available to the team is limited, so device coverage is by screen size and Android version rather than by manufacturer. • Emulators are not used for the camera, GPS and permission cases, because their behaviour differs from that of a real handset in exactly those areas. • Configuration changes on the deployed system are made only with the team's agreement and are reverted afterwards.

4. Deliverables

The following artifacts are produced where the corresponding testing activity is performed. Relevant summaries and evidence are included in this report.

- Automated test execution reports
- Test coverage reports for the four suites
- Continuous integration results
- API test collection and Swagger UI execution results
- Access control matrix and its results
- Security scan reports
- Load test reports and graphs
- Accessibility and usability study results
- Test case log, with pass and fail status

4.1 Test Evaluation Summaries

Automated test suites

The platform's four components each carry an automated test suite, which runs locally and on every push and pull request through GitHub Actions. The results below were produced on a Windows 11 workstation with Python 3.11.9 and Node.js 24.

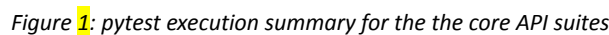
Suite	Framework	Files	Executed	Passed	Failed	Skipped
Core API	pytest 8.4.2, pytest-asyncio 1.4.0	88	1,054	1,043	0	11
Face verification	pytest 8.4.2	24	233	231	0	2
Web dashboard	Vitest 4.1.10, Testing Library	30	146	146	0	0
Android application	Jest 29.7, jest-expo 57.0.3	64	517	517	0	0
Total		206	1948	1937	0	11

Table 4: Automated Test Suite Execution Summary

Core API

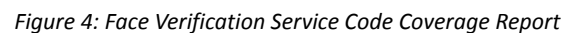
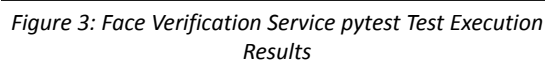
The suite tests service, route, and repository behaviour across the main attendance modules. It includes QR session generation and validation, lecturer session lifecycle, geofence

In the latest CI run, 1,043 tests passed and 11 were skipped. The skipped database integration tests require a separately configured local PostgreSQL database. They cover the attendance lifecycle, geofencing, active session discovery, and QR evidence. Some also test concurrency. Because they did not run in standard CI, their outcomes remain unverified in that run.



In the local run, 231 tests passed, two were skipped, and overall statement coverage was 88 per cent. The suite's largest area is liveness evidence validation, with 42 tests covering valid evidence and the rejection of malformed, wrongly versioned, wrongly methoded, failed, expired, and future-dated evidence before face comparison. The remaining tests cover verification routes and readiness checks, embedding similarity and comparison, encrypted

The suite tests the logic surrounding the face model. It does not measure how accurately the model distinguishes one person from another. Face recognition accuracy should be measured separately using genuine and mismatched face pairs at the configured threshold.



The latest CI run passed 146 tests across 30 files. A separate local coverage run reported 59.7 per cent statement coverage, 59.5 per cent branch coverage, and 62.2 per cent line coverage. Coverage varies across the application: shared interface components reached 95.6 per cent statement coverage and server actions reached 76.8 per cent. Lower results were recorded for the authentication data access layer (6.3 per cent), QR batch void control (3.6 per cent), lecturer API client (29.3 per cent), academic data workspace (32.0 per cent), and lecturer service (40.2 per cent). These lower-coverage areas should be prioritised for further automated and manual testing.

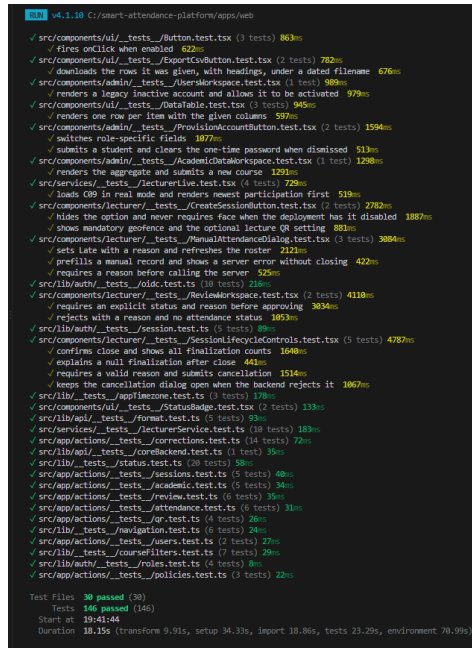


Figure 5: Web Application Vitest Test Execution Summary

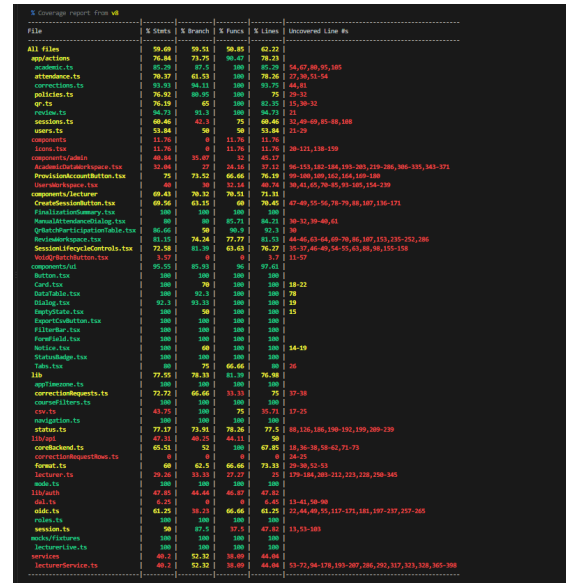


Figure 6: Web Application Vitest Code Coverage Report

Android application

The Android application has strong automated test coverage across the main functional areas, with all 517 tests passing. The overall line coverage of 85.67% indicates that most of the application code is exercised by the test suite. Statement and function coverage are also above 83%, which suggests good coverage of the implemented logic. The lower branch coverage of 77.13% shows that some alternative conditions and decision paths are not yet fully tested, so additional negative and edge-case tests could improve confidence further.

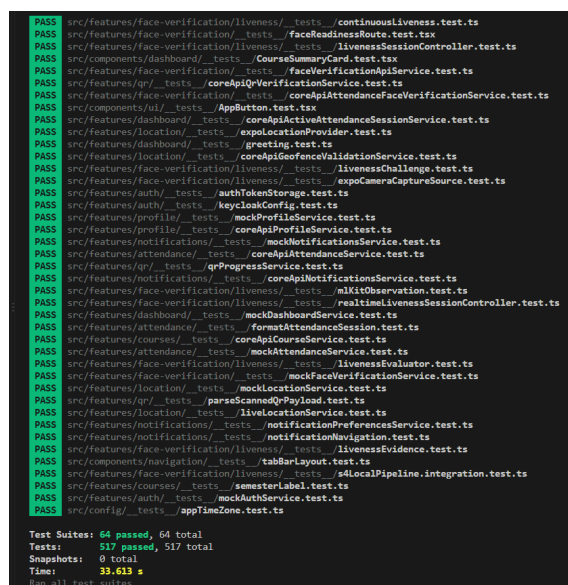


Figure 7: Android Application Jest Test Execution Summary

[illegible]

Figure 8: Android Application Jest Code Coverage Report

Continuous integration

The repository uses five main GitHub Actions workflows: Backend CI for the two Python services, Web CI for the web application, Mobile CI for the Android application, Deployment CI for deployment verification, and Pilot HTTPS uptime for monitoring the deployed system. The Backend, Web, Mobile and Deployment workflows run automatically on the configured repository events, such as pushes and pull requests, while the uptime workflow runs on a scheduled basis. These automated checks help identify regressions and deployment problems early rather than waiting until the end of the project.

Actions

New workflow

All workflows

Backend CI

Copilot

Copilot cloud agent

Deployment CI

Mobile CI

Pilot HTTPS uptime

Web CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

642 workflow runs

Workflow

Event

Status

Branch

Actor

✓ Pilot HTTPS uptime

Pilot HTTPS uptime #20: Scheduled

main

42 minutes ago

8s

✓ Merge pull request #109 from Smart-Attendance-Group-27/deploy...

Mobile CI #269: Commit [7eaf301](#) pushed by [Manushan-hasanika](#)

main

Today at 2:14 PM

1m 15s

✓ Merge pull request #109 from Smart-Attendance-Group-27/deploy...

Backend CI #92: Commit [7eaf301](#) pushed by [Manushan-hasanika](#)

main

Today at 2:14 PM

39s

✓ Merge pull request #109 from Smart-Attendance-Group-27/deploy...

Web CI #187: Commit [7eaf301](#) pushed by [Manushan-hasanika](#)

main

Today at 2:14 PM

1m 12s

✓ Merge pull request #109 from Smart-Attendance-Group-27/deploy...

Deployment CI #65: Commit [7eaf301](#) pushed by [Manushan-hasanika](#)

main

Today at 2:14 PM

5m 23s

✓ Deploy application PostgreSQL on the pilot VPS

Backend CI #91: Pull request #109 opened by [itmanushan](#)

deployment/vps-application-

Today at 2:11 PM

43s

✓ Deploy application PostgreSQL on the pilot VPS

Web CI #186: Pull request #109 opened by [itmanushan](#)

deployment/vps-application-

Today at 2:11 PM

1m 5s

Figure 9: GitHub Actions Continuous Integration Workflow Results

API testing

Manual API testing was carried out using the FastAPI Swagger UI in the local development environment. The API was tested with invalid, incomplete, duplicate and unauthorized requests to verify input validation, authentication and role-based access control. The tested endpoints correctly rejected requests with missing required fields, malformed JSON and invalid field values using HTTP 422 responses. A duplicate course code was rejected with HTTP 409 Conflict. Requests without an access token were rejected with HTTP 401 Unauthorized, while an authenticated user without the required administrator role received HTTP 403 Forbidden. These results show that the tested API endpoint correctly validates request data and enforces authentication and authorization rules.

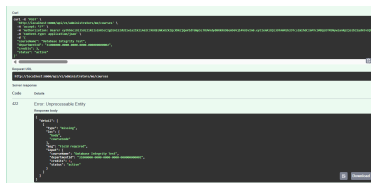


Figure 10: Swagger UI Validation Test - Missing Required Field Returned HTTP 422

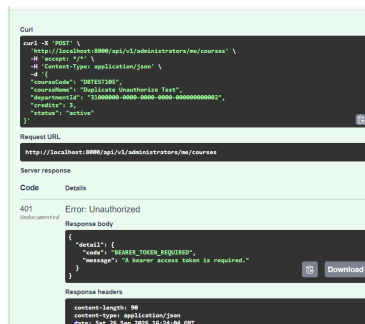


Figure 11: Swagger UI Authentication Test - Missing Bearer Token Returned HTTP 401

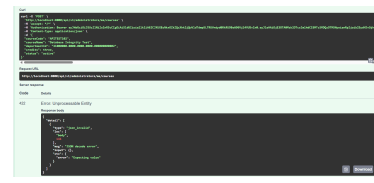


Figure 12: Swagger UI Invalid JSON Validation Test

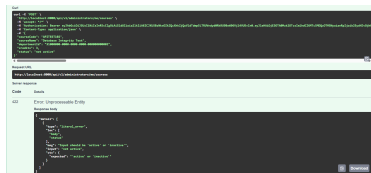


Figure 13: Swagger UI Invalid Status Validation Test

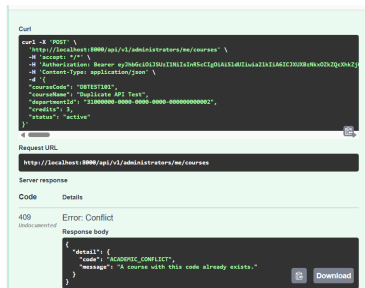


Figure 14: Swagger UI Duplicate Course Test

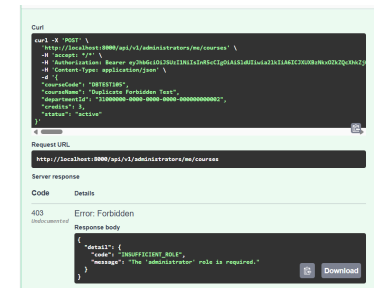


Figure 15: Swagger UI Role-Based Access Test

Security testing

Security testing covered authentication, role-based access control, and dependency vulnerabilities. A role matrix was evaluated across 11 critical API endpoints for Student, Lecturer, and Administrator access. All expected permissions matched the implemented behaviour. Unauthenticated requests were rejected with 401 Unauthorized, and authenticated users with the wrong role were rejected with 403 Forbidden.

No access-control mismatches were identified in the tested endpoints.

Endpoint	Method	Student	Lecturer	Admin	Result
/api/v1/me	GET	Allow	Allow	Allow	Pass
/api/v1/attendance-sessions/{session_id}/check-in	POST	Allow	Deny	Deny	Pass
/api/v1/attendance-sessions/{session_id}/geofence-attempts	POST	Allow	Deny	Deny	Pass
/api/v1/qr-sessions/{qr_session_id}/verify	POST	Allow	Deny	Deny	Pass
/api/v1/attendance-sessions/{session_id}/face-verifications	POST	Allow	Deny	Deny	Pass
/api/v1/lecturers/me/attendance-sessions	POST	Deny	Allow	Deny	Pass
/api/v1/lecturers/me/attendance-sessions/{session_id}/close	POST	Deny	Allow	Deny	Pass
/api/v1/lecturers/me/attendance-sessions/{session_id}/students/{student_id}/attendance	PUT	Deny	Allow	Deny	Pass
/api/v1/administrators/me/correction-requests/{request_id}/decision	POST	Deny	Deny	Allow	Pass
/api/v1/administrators/me/users	POST	Deny	Deny	Allow	Pass
/api/v1/administrators/me/reference-faces	GET	Deny	Deny	Allow	Pass

Table 5: Role-Based Access Control Matrix for Critical API Endpoints

Accessibility and usability

Accessibility and usability testing checks whether the web and mobile applications are easy to use and accessible to different users. Tools such as Google Lighthouse can be used to find problems such as missing labels, poor colour contrast, and accessibility issues.

Manual checks are also carried out for keyboard navigation, clear error messages, readability, screen navigation, and completing common tasks. For the Android application, testing can also include permission handling and TalkBack.

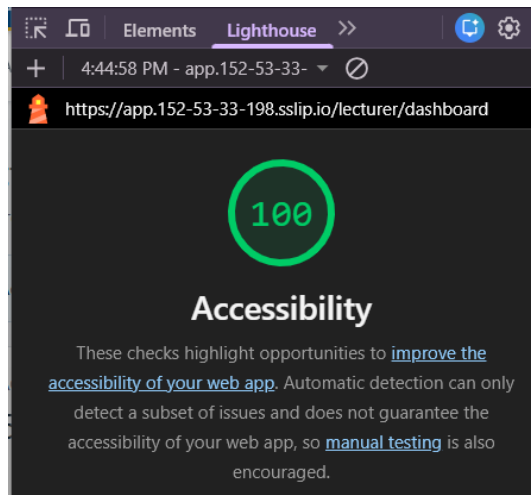


Figure 16: Lighthouse Accessibility Audit for the Lecturer Dashboard — Score 100

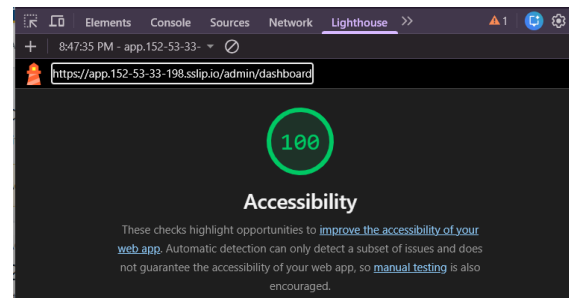


Figure 17: Lighthouse Accessibility Audit for the Admin Dashboard — Score 100

4.2 Reporting on Test Coverage

Coverage is reported in two senses: the proportion of the source code executed by the automated suites, and the proportion of the requirements exercised by at least one test case.

Code coverage is measured by pytest-cov for the two Python services, by the v8 provider for the web application, and by Jest for the Android application. It is produced on every local run and is reviewed after any change that adds a module. The figures are reported in 4.1. Code coverage is treated as a diagnostic rather than a target: a high figure does not prove the tests assert anything useful, but a low figure reliably identifies code no test has ever executed, and those areas were prioritised for manual testing.

Requirements coverage is recorded in a traceability matrix listing every functional and non-functional requirement against the test cases that exercise it, so that a requirement with no test is visible.

Type	Requirement	Test case IDs	Technique	Status
FR	Authenticate a user and return their identity	test_returns_the_signed_in_application_user; test_rejects_an_expired_token	Automated route tests	Passed
FR	Lecturer creates an attendance session	test_create_session; test_create_session_invalid_schedule_returns_422	Automated route tests	Passed

FR	Validate student location during check-in	test_eligible_student_pass_is_stored_and_remains_in_progress; test_a_failed_geofence_reading_does_not_ask_check_in_to_run	Automated service tests	Passed
FR	Verify a student's QR code	test_verify_qr_session_accepts_correct_qr_value; test_verify_qr_session_returns_expired_for_expired_token	Automated service tests	Passed
FR	Verify a student's face with liveness evidence	test_valid_liveness_allows_face_verification_through_backend_chain; test_missing_liveness_is_rejected_through_the_backend_chain	Automated route tests	Passed
FR	Finalize attendance when a session closes	test_closing_decides_every_students_attendance_from_the_real_evidence; test_closing_twice_is_refused_and_writes_nothing_more	PostgreSQL integration tests	Skipped in standard CI
FR	Lecturer records manual attendance	test_sets_the_status_and_returns_the_manual_record; test_a_student_cannot_set_attendance	Automated route tests	Passed
FR	Administrator reviews correction requests	test_pending_request_can_be_approved_and_is_audited; test_lecturer_cannot_list_or_decide	Automated service and route tests	Passed
NFR	Enforce role-based access control	Role matrix for 11 critical endpoints in Section 4.1; relevant negative tests above	API access-control testing	Passed for the listed endpoints
NFR	Meet response-time targets	k6 scenarios in Section 3.1.4	Performance testing	

Table 6: Traceability Matrix for Selected Functional and Non-Functional Requirements

Frequency of reporting. Automated suite results are produced on every push through continuous integration and are reviewed daily during development. Coverage reports are produced weekly and after any significant feature change. Manual test case results are

recorded as they are executed. Security scanning is repeated after any dependency update. Load and performance testing is repeated after any change to the check-in path, since that is the path under load.

Report contents. Each test report records the date, the tester, the component and test case identifier, the number of test cases executed, the number passed and failed, the pass and fail percentages, the environment used, and comments. Reports are produced for both successful and unsuccessful runs, since a run with no failures is itself the evidence that a previously fixed defect has not returned.

5. Risks, Dependencies, Assumptions, and Constraints

Risk	Likelihood / Impact	Mitigation Strategy	Contingency (risk is realized)
Testing against the deployed system corrupts pilot data, because the deployment writes to the live database	Medium / High	A dedicated test course, test session and test student accounts are used for all deployed testing. A database backup is taken before any destructive or high-volume test. The team is informed of each test window.	Restore the affected tables from the most recent backup; remove the generated test rows by session identifier.
Load testing saturates the single VPS and interrupts the pilot	Medium / High	Run tests during an agreed window, increase load in steps, and stop at predefined error-rate or resource-use limits.	Stop the load generator, restart affected containers if needed, and check attendance data for duplicates or incomplete records. Report the highest load successfully tested on the VPS.
The face-verification service is the bottleneck and cannot support a full class	Medium / High	Test the face service separately with a burst of requests to see how many it can handle before testing the complete check-in process.	If testing shows that the face service cannot handle a full class, report the measured capacity and recommend a queue, another service instance, or a larger host.

Only a few physical Android devices are available, so device coverage is narrow	High / Medium	Cover by screen size and Android version rather than by manufacturer, and borrow a second device from the team where possible.	Record the tested configurations explicitly and state the untested ones as a limitation of the results.
Test data proves inadequate, particularly for face accuracy testing, where varied faces and lighting are needed	Medium / Medium	Collect face captures from consenting team members under several lighting conditions, with and without spectacles, before the accuracy test begins.	If sufficient varied test data cannot be collected, report the sample size and conditions tested, and limit the accuracy claim to those conditions.
Biometric test data creates a privacy obligation of its own	Medium / High	Use only the faces of consenting team members. Store captures outside the repository. Delete them when testing is complete. Redact identifying images from the report.	Delete the data immediately and record the deletion.
An active security scan disrupts the deployed system or creates spurious data	Medium / Medium	Run only passive scanning against the deployed system. Run active scanning against a local copy, after a backup.	Stop the scan, check the affected service and data, and remove test rows. Restore the local test database from backup only if needed.
Keycloak becomes unavailable during testing, blocking all authenticated tests	Low / High	Keep the realm backup and restore scripts verified and current. Obtain the three test accounts early so a later outage does not block test design.	Pause authenticated tests, investigate and restore Keycloak availability, then verify the test accounts before resuming. Use a realm backup only if realm data was lost or corrupted.
A high-severity defect is found late, leaving no time to fix it	Medium / High	Test security and the check-in path early, before interface polish.	Record and prioritise the defect. Fix and retest it if possible; otherwise, state clearly in the conclusions that it remains unresolved and describe its impact.

Dependencies. Testing depends on the deployed environment being available; working Student, Lecturer and Administrator test accounts in Keycloak; a physical Android device; a local test stack with PostgreSQL for database integration and failover tests; approximately 20–25 usability participants; and agreed test windows for load testing and scans of the deployed system.

Assumptions. The deployed version is assumed to correspond to the repository version being evaluated; this should be checked against the deployed commit. Test data is assumed to be separate from real student records; this should be confirmed before testing. The deployed geofence radius, face threshold and QR refresh interval should also be confirmed. Face images will be used only with participants' consent.

Constraints. The release supports Android only, so iOS is outside the test scope. Performance results are specific to the tested VPS and its connected services, including Supabase. Physical power interruption cannot be tested on the hosted VPS, so failure injection is planned on a local test stack. Recruiting and scheduling approximately 20-25 participants may constrain the usability study; the report will state the number who actually participated.

6. References

- [1] pytest, “pytest documentation.” <https://docs.pytest.org/>. [Accessed: Aug. 10, 2026].
- [2] Vitest, “Vitest documentation.” <https://vitest.dev/guide/>. [Accessed: Aug. 10, 2026].
- [3] Jest, “Jest documentation.” <https://jestjs.io/docs/>. [Accessed: Aug. 10, 2026].
- [4] Testing Library, “Testing Library documentation.” <https://testing-library.com/docs/>.
- [5] Grafana Labs, “API load testing,” *Grafana k6 Documentation*. [Online]. Available: <https://grafana.com/docs/k6/latest/testing-guides/api-load-testing/>. [Accessed: Sep. 27, 2026].
- [6] Grafana Labs, “Thresholds,” *Grafana k6 Documentation*. [Online]. Available: <https://grafana.com/docs/k6/latest/using-k6/thresholds/>. [Accessed: Sep. 27, 2026].
- [7] Grafana Labs, “Web dashboard,” *Grafana k6 Documentation*. [Online]. Available: <https://grafana.com/docs/k6/latest/results-output/web-dashboard/>. [Accessed: Sep. 27, 2026].
- [8] OWASP Foundation, “OWASP Top 10:2021.” <https://top10.owasp.org/2021/>. top10.owasp.org.
- [9] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1.” <https://www.w3.org/TR/WCAG21/>.
- [10] Google, “Lighthouse,” Chrome for Developers. <https://developer.chrome.com/docs/lighthouse/>. [Accessed: Sep. 25, 2026].